

MC521 - Relatório VI

Júlio da Silva Telles

20 de Junho de 2026

1 I - MCM (Aizu - ALDS1_10_B)

O problema consiste em encontrar o número mínimo de multiplicações escalares necessárias para computar o produto de uma cadeia de N matrizes.

1.1 Solução

Podemos resolver este problema utilizando Programação Dinâmica (DP) em um formato *Bottom-Up*, com complexidade de tempo de $\mathcal{O}(N^3)$. O algoritmo segue os seguintes passos:

1. **Inicialização do Estado:** Preenchemos a diagonal principal da nossa matriz de custos m com zeros ($m[i][i] = 0$), pois o custo de multiplicar uma única matriz (sem outras matrizes para multiplicar) é nulo.
2. **Iteração por Subcadeias:** Calculamos as respostas para cadeias de matrizes cada vez maiores. O loop mais externo u define o tamanho da subcadeia analisada, enquanto o loop i define onde essa subcadeia começa, determinando também o índice final j .
3. **Otimização do Ponto de Corte:** Para o intervalo de matrizes de i a j , testamos todos os pontos de quebra possíveis k . Calculamos o custo da operação como a soma do custo ótimo da metade esquerda, da metade direita, e o custo final de multiplicar as duas matrizes resultantes ($d[i] * d[k + 1] * d[j + 1]$). Atualizamos a resposta $m[i][j]$ com o menor custo obtido.

2 D - Clear the String (CodeForces - 1132F)

O problema consiste em encontrar o número mínimo de operações necessárias para apagar uma string inteira, sabendo que em cada operação podemos remover uma substring contígua de caracteres idênticos.

2.1 Solução

Podemos modelar o problema com Programação Dinâmica em intervalos (*Interval DP*) utilizando a abordagem *Top-Down* com *Memoization*, resultando em uma complexidade de tempo de $\mathcal{O}(N^3)$. O algoritmo segue os seguintes passos:

1. **Compressão da String:** Ao ler a string original, ignoramos e não armazenamos letras adjacentes iguais na nova variável `s` (por exemplo, "aab" vira "ab"). Isso simplifica a DP, já que apagar "aa" de uma vez tem o mesmo custo e regra que apagar apenas "a".
2. **Interval DP (*Memoization*):** A função recursiva `clear(l, r)` retorna o custo mínimo para esvaziar o intervalo entre os índices l e r . Usamos as matrizes `visited` e `memo` para guardar os resultados de subproblemas já resolvidos e evitar recálculos. Se o índice da esquerda passar o da direita ($l > r$), a base da recursão retorna 0.
3. **Transição de Estados:** Para qualquer intervalo analisado, assumimos inicialmente a pior hipótese: deletar o caractere mais à esquerda isoladamente, custando $1 + \text{clear}(l + 1, r)$. Em seguida, iteramos no restante do intervalo buscando algum caractere no índice i idêntico ao inicial (`s[l] == s[i]`). Se houver igualdade, calculamos o custo de apagar o "miolo" (`clear(l+1, i-1)`) para juntar os caracteres iguais, somado ao custo do restante do grupo fundido (`clear(i, r)`). Retornamos o mínimo entre todas as possibilidades.